

Technische Dokumentation

Frontend der ILSE-Plattform

Überblick über Seiten, Komponenten und Hilfsmodule des Frontends

1. Einleitung

Dieses Dokument beschreibt den Aufbau des Frontends der ILSE-Plattform. Die Anwendung ist mit React, TypeScript und Vite umgesetzt und bildet die Web-Oberfläche zum in einem separaten Dokument beschriebenen Backend.

Die Dokumentation richtet sich an alle Projektbeteiligten, die einen Überblick über die vorhandenen Seiten und Komponenten benötigen, unabhängig vom technischen Hintergrund. Jede Komponente bzw. jedes Hilfsmodul wird einheitlich beschrieben: Zweck, Props bzw. Methoden und Zusammenhang mit anderen Komponenten.

1.1 Technischer Aufbau

Das Projekt basiert auf folgenden Technologien:

Technologie	Verwendung
React 19	Aufbau der Benutzeroberfläche über Komponenten
TypeScript	Typisierte Programmiersprache, Basis für alle .tsx/.ts-Dateien
React Router 7	Clientseitiges Routing zwischen den Seiten
Vite	Build-Tool und Entwicklungsserver
SCSS	Styling der Komponenten (je eine .scss-Datei pro Komponente)

Der Quellcode liegt unter src/ und ist in zwei Bereiche gegliedert:

- components/ – alle Seiten und UI-Komponenten, in Unterordnern nach Themenbereich sortiert (z. B. Login, Profile, RoomView).
- lib/ – Hilfsfunktionen, TypeScript-Typen und die Anbindung an die API, unabhängig von der Darstellung.

Der Einstiegspunkt main.tsx bindet die Komponente App ein, die über React Router die Routen auf die jeweiligen Seiten-Komponenten abbildet (siehe Kapitel 2).

1.2 Beschreibungsschema

Jede Komponente wird nach folgendem einheitlichen Schema beschrieben:

- Zweck: wofür die Komponente zuständig ist.
- Props: die Werte, die eine Komponente von ihrer übergeordneten Komponente erhält, mit Typ und Bedeutung.
- Verhalten: die wichtigsten Abläufe innerhalb der Komponente, sofern vorhanden.

Hilfsmodule aus lib/ werden stattdessen als Zweck und Methoden-Tabelle beschrieben, da sie keine Benutzeroberfläche darstellen.

2. Seiten (Routen)

Die Komponente App definiert über React Router alle Seiten der Anwendung. Für jeden vorhandenen Lernraum wird zur Laufzeit automatisch eine eigene Route erzeugt.

Pfad	Seiten-Komponente	Beschreibung
/	LandingPage	Startseite
/about	WhoWeArePage	Über-uns-Seite mit Feedback-Formular
/login	LoginPage	Anmeldung und Registrierung
/profile	ProfilePage	Profilseite (eigener Fortschritt bzw. Schülerverwaltung)
/rooms	RoomsPage	Übersicht aller Lernräume
/rooms/{roomName}	RoomViewPage	Inhalt und Aufgaben eines einzelnen Lernraums, für jeden Raumnamen automatisch erzeugt
*	InvalidPage	Fehlerseite für unbekannte Pfade

Hinweis: Die Raum-Routen werden aus `getRoomNames()` (`lib/dataHandler`) erzeugt. Ändert sich die Raumliste, passen sich die Routen automatisch an.

LandingPage

Zweck: Startseite der Anwendung. Setzt den Seitentitel und zeigt Header, Einstiegsbereich, Themenkarten und Footer an. Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus `lib/` (z. B. `Session-Storage` oder `Mock-Daten`).

Verhalten

- Rendert `InfoPage` (Einstiegstext) und `CardsPage` (Themenkarten) zwischen Header und Footer.

WhoWeArePage

Zweck: Informationsseite über das Projektteam mit Feedback-Formular.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus `lib/` (z. B. `Session-Storage` oder `Mock-Daten`).

Verhalten

- Rendert `AboutText` und `AboutFeedback` unterhalb des Headers.

LoginPage

Zweck: Seite für Anmeldung und Registrierung, wechselt zwischen beiden Formularen.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus `lib/` (z. B. `Session-Storage` oder `Mock-Daten`).

Verhalten

- Verwaltet, ob das Login- oder das Registrierungsformular angezeigt wird.
- Nach erfolgreicher Registrierung wird automatisch auf das Login-Formular umgeschaltet und eine Erfolgsmeldung angezeigt.

ProfilePage

Zweck: Zeigt die Profildaten des angemeldeten Benutzers an; bei Lehrkräften zusätzlich die Schülerverwaltung.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus `lib/` (z. B. `Session-Storage` oder `Mock-Daten`).

Verhalten

- Lädt die Profildaten über `getProfileData()` beim Aufruf der Seite.
- Zeigt `StudentDashboard` nur an, wenn `userType` gleich "teacher" ist und `studentsInfo` vorhanden ist.

RoomsPage

Zweck: Übersichtsseite aller verfügbaren Lernräume.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus `lib/` (z. B. `Session-Storage` oder `Mock-Daten`).

Verhalten

- Rendert dieselbe `CardsPage`-Komponente wie die Startseite, umrahmt von einer eigenen Überschrift.

RoomViewPage

Zweck: Zeigt den Inhalt eines einzelnen Lernraums mit allen zugehörigen Themen und Aufgaben an und berechnet den Fortschritt in diesem Raum.

Prop	Typ	Bedeutung
roomName	string	Name des anzuzeigenden Raums
thumbnailSrc	string null	Vorschaubild des Raums für den Kopfbereich

Verhalten

- Lädt den Rauminhalt über `getRoomContent(roomName)`.
- Führt für jedes Thema Buch, ob es abgeschlossen ist, und berechnet daraus den prozentualen Fortschrittsbalken.
- Meldet ein abgeschlossenes Thema über `sendTopicStatus` an die API, sofern der Benutzer angemeldet ist.

InvalidPage

Zweck: Wird für alle nicht vorhandenen Routen angezeigt (404-Seite).

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Verhalten

- Setzt den Seitentitel und zeigt Header sowie ErrorPage an.

3. Layout-Komponenten

Header und Footer werden auf nahezu jeder Seite eingebunden und sorgen für Navigation und Konsistenz.

Header (AppHeader)

Zweck: Kopfbereich mit Logo, Navigation, Login-/Logout-Button und Farbmodus-Umschalter.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Verhalten

- Ermittelt den Anmeldestatus über `isLoggedIn()` und zeigt entsprechend "Anmelden" oder "Abmelden" an.
- Übernimmt den bevorzugten Farbmodus zunächst aus dem Session-Storage, sonst aus der Systemeinstellung des Browsers, und speichert Änderungen wieder im Session-Storage.
- Blendet bei Klick auf den Menü-Button eine Navigationsfläche mit Schnellzugriffen und der Liste aller Lernräume ein.

Footer (AppFooter)

Zweck: Fußbereich mit Kurzinfo zur Plattform und Links zu allen Lernräumen.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Verhalten

- Lädt die Raumnamen über `getRoomNames()` und erzeugt daraus die Navigationslinks.

4. Startseite (Landing)

InfoPage

Zweck: Einstiegsbereich der Startseite mit Begrüßungstext und Illustration.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

CardsPage

Zweck: Zeigt alle Lernräume als Kartenübersicht an. Wird sowohl auf der Startseite als auch auf der Raum-Übersichtsseite verwendet.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Verhalten

- Lädt die Raumlite über `getRoomListItems()` und rendert für jeden Raum eine Card-Komponente.
- Zeigt bei einem Fehler beim Laden eine Fehlermeldung anstelle der Karten an.

Card

Zweck: Einzelne Themenkarte mit Bild, Titel und Kurzbeschreibung eines Lernraums.

Prop	Typ	Bedeutung
topicInfo	roomListItem	Name, Vorschaubild und Beschreibung des Raums

Verhalten

- Navigiert bei Klick zur zugehörigen Raumseite (`/rooms/{name}`).

5. Anmeldung (Login / Signup)

Login

Zweck: Login-Formular mit Eingabefeldern für Benutzername und Passwort.

Prop	Typ	Bedeutung
registrSuccess	boolean	Zeigt eine Erfolgsmeldung an, wenn zuvor eine Registrierung abgeschlossen wurde

Verhalten

- Prüft beim Absenden, ob beide Felder ausgefüllt sind, und zeigt sonst eine Fehlermeldung je Feld an.
- Ruft bei gültiger Eingabe `sendLoginData` auf und speichert das erhaltene Token über `setLoginState`.
- Leitet bei erfolgreicher Anmeldung zur Profilseite weiter.

Signup

Zweck: Registrierungsformular mit Benutzername, Passwort und Passwort-Bestätigung.

Prop	Typ	Bedeutung
setLogin	() => void	Wird nach erfolgreicher Registrierung aufgerufen, um zum Login-Formular zu wechseln

Verhalten

- Prüft, ob alle drei Felder ausgefüllt sind und beide Passwort-Felder übereinstimmen.
- Ruft bei gültiger Eingabe `sendSignupData` auf.

6. Profil-Bereich

Die Profilseite zeigt für alle Benutzer den eigenen Fortschritt an. Lehrkräfte erhalten zusätzlich eine Schülerverwaltung.

6.1 Eigenes Profil

ProfileInfo

Zweck: Fasst die eigenen Profildaten, die Kontoeinstellungen und den Raum-Fortschritt zu einem Block zusammen.

Prop	Typ	Bedeutung
content	profileContent	Alle Profildaten des Benutzers
updateIcon	(newIcon: profileIconElement) => void	Aktualisiert das Profilbild nach einer Änderung

Verhalten

- Rendert ProfileCard, AccountControl und für jeden Raum eine RoomsStatus-Anzeige.

ProfileCard

Zweck: Zeigt Profilbild, Benutzername, Level und Level-Fortschrittsbalken an.

Prop	Typ	Bedeutung
profileInfo	profileContent	Anzuzeigende Profildaten

Verhalten

- Setzt Hintergrundfarbe des Profilbilds und Breite des Fortschrittsbalkens direkt über eine Referenz auf das jeweilige Element.

AccountControl

Zweck: Bietet Optionen zum Ändern des Profilbilds, Ändern des Passworts und – für Lehrkräfte – Löschen des Accounts.

Prop	Typ	Bedeutung
userType	string	Rolle des Benutzers ("teacher" schaltet die Löschoption frei)
userIcon	profileIconElement	Aktuelles Profilbild
updateIcon	(newIcon: profileIconElement) => void	Callback zum Übernehmen eines neuen Profilbilds

Verhalten

- Öffnet je nach gewählter Option UserIconAlertBox, PasswordAlertBox oder AlertBox und verarbeitet deren Ergebnis über manageOwnAcc.

RoomsStatus

Zweck: Einzelne Fortschrittsanzeige (Name und Fortschrittsbalken) für einen Lernraum. Wird sowohl für das eigene Profil als auch für Schülerprofile verwendet.

Prop	Typ	Bedeutung
roomInfo	profileRoomsProgress	Raumname und Fortschritt in Prozent

6.2 Schülerverwaltung (nur für Lehrkräfte)

StudentDashboard

Zweck: Rahmen für die Schülerverwaltung, bestehend aus Verwaltungsoptionen und Schülerliste.

Prop	Typ	Bedeutung
studentsInfo	profileStudentsInfo[] null	Liste aller Schüler mit ihrem Fortschritt

StudentControls

Zweck: Bietet die Möglichkeit, neue Schüleraccounts anzulegen.

Prop	Typ	Bedeutung
studentsInfo	profileStudentsInfo[]	Aktuelle Schülerliste (wird an dieser Stelle nicht weiter ausgewertet)

Verhalten

- Blendet bei Klick auf den Button AddStudentControl ein.

AddStudentControl

Zweck: Formular zum Anlegen einer bestimmten Anzahl neuer Schüleraccounts.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Verhalten

- Erlaubt die Eingabe einer Anzahl zwischen 1 und 100.
- Ruft bei Bestätigung addNewStudents auf und zeigt das Ergebnis über NewStudentsList an.

NewStudentsList

Zweck: Zeigt die Liste der soeben neu angelegten Schüler mit Benutzername und vorläufigem Passwort an.

Prop	Typ	Bedeutung
students	createdStudentsInfo null	Die neu angelegten Schüler-Zugangsdaten

StudentList

Zweck: Liste aller Schüler einer Lehrkraft.

Prop	Typ	Bedeutung
studentsInfo	profileStudentsInfo[]	Alle Schüler mit ihrem jeweiligen Fortschritt

Verhalten

- Rendert für jeden Schüler eine StudentInfoElement-Komponente.

StudentInfoElement

Zweck: Einzelner, aufklappbarer Schülereintrag mit Namen, Profilbild und – bei Bedarf – Raum-Fortschritt und Verwaltungsoptionen.

Prop	Typ	Bedeutung
studentInfo	profileStudentsInfo	Name, Profilbild und Fortschritt des Schülers

Verhalten

- Blendet bei Klick den Fortschritt je Raum sowie StudentControlElement ein oder aus.

StudentControlElement (StudCtrlElement.tsx)

Zweck: Bietet für einen einzelnen Schüler die Optionen "Account löschen" und "Passwort zurücksetzen" mit Sicherheitsabfrage.

Prop	Typ	Bedeutung
studentName	string	Name des betroffenen Schülers

Verhalten

- Zeigt vor dem Löschen bzw. Zurücksetzen jeweils eine AlertBox zur Bestätigung an.
- Zeigt nach einem Passwort-Reset das neu vergebene, temporäre Passwort an.

7. Lernräume und Aufgaben (RoomView)

Dieser Bereich zeigt den Inhalt eines Lernraums an: eine Liste von Themen, die jeweils eine Aufgabe und einen Beschreibungstext enthalten.

RoomTopic

Zweck: Ein einzelnes, aufklappbares Thema innerhalb eines Raums mit Aufgabe, optionalem Hilfetext und Abgabe-Button.

Prop	Typ	Bedeutung
index	number	Position des Themas innerhalb des Raums
topic	roomTopic	Themendaten inklusive Aufgabe und Beschreibungstext
updateRoomProgress	(index, topicID, isCompleted) => void	Meldet den Bearbeitungsstatus an RoomViewPage zurück
updateComplete	number	Zähler, über den RoomViewPage das Ende der Auswertung signalisiert

Verhalten

- Übernimmt automatisch den Status "abgeschlossen", falls die Aufgabe laut Daten bereits erledigt wurde.
- Löst bei Klick auf "Abgeben" die Auswertung der Aufgabe über TopicExercise aus und meldet das Ergebnis an RoomViewPage.

TopicDescription

Zweck: Zeigt den erklärenden Hilfetext zu einem Thema an, wenn dieser eingeblendet wurde.

Prop	Typ	Bedeutung
description	string	Anzuzeigender Hilfetext

TopicExercise

Zweck: Wählt anhand des Aufgabentyps die passende Aufgaben-Komponente aus und reicht die Auswertungs-Signale weiter.

Prop	Typ	Bedeutung
exercise	roomTopicExercise	Aufgabendaten inklusive Typ und Fragen
checkSignal	number	Signal von RoomTopic, die Antwort zu prüfen
checkDoneSignal	() => void	Rückmeldung, dass die Prüfung abgeschlossen ist
setExerciseState	(state: boolean) => void	Meldet, ob die Aufgabe richtig gelöst wurde

Verhalten

- Unterscheidet aktuell zwischen den Aufgabentypen "multiple-choice" und "linking".

Hinweis: Für die Aufgabentypen Drag-and-Drop und anklickbares Bild existieren eigene Komponenten (siehe unten), sie werden hier jedoch noch nicht eingebunden.

MultipleChoiceExercise

Zweck: Multiple-Choice-Aufgabe mit mehreren ankreuzbaren Antwortoptionen.

Prop	Typ	Bedeutung
question	exerciseQuestion	Frage- und Antworttext samt Lösungen
checkSignal	number	Signal, die aktuelle Auswahl zu prüfen
checkDoneSignal	() => void	Rückmeldung nach abgeschlossener Prüfung
setExerciseState	(state: boolean) => void	Meldet, ob alle Antworten korrekt gesetzt sind

Verhalten

- Gilt als korrekt gelöst, wenn der Status jeder einzelnen Antwortoption (siehe SingleChoiceElement) auf "richtig" steht.

SingleChoiceElement (SingleChoiceElement.tsx)

Zweck: Einzelne ankreuzbare Antwortoption innerhalb einer Multiple-Choice-Aufgabe.

Prop	Typ	Bedeutung
answer	exerciseAnswer	Text und Lösungsinformation dieser Antwortoption
checkAnswerSignal	number	Signal, den aktuellen Ankreuzstatus zu prüfen
notifyUserSignal	boolean	Wird aktuell übergeben, aber nicht ausgewertet
index	number	Kennung dieser Antwortoption
reportAnswerState	(index, state) => void	Meldet das Prüfungsergebnis an MultipleChoiceExercise zurück
answerState	number	Übergebener Anfangsstatus

Verhalten

- Vergleicht bei einem Prüfsignal den eigenen Ankreuzstatus mit isCorrect und meldet richtig/falsch zurück.

LinkingExercise

Zweck: Zuordnungsaufgabe, bei der zwei zusammengehörende Einträge nacheinander angeklickt werden.

Prop	Typ	Bedeutung
question	exerciseQuestion	Die zuzuordnenden Einträge
checkSignal	number	Wird entgegengenommen, aktuell nicht ausgewertet
checkDoneSignal	() => void	Wird entgegengenommen, aktuell nicht aufgerufen
setExerciseState	(state: boolean) => void	Wird entgegengenommen, aktuell nicht aufgerufen

Verhalten

- Prüft automatisch nach der Auswahl von zwei Einträgen, ob diese zusammengehören, und blendet richtige Paare aus bzw. setzt falsche Paare zurück.

LinkingElement

Zweck: Einzelner anklickbarer Eintrag innerhalb einer Zuordnungsaufgabe.

Prop	Typ	Bedeutung
id	number	Position innerhalb der Liste
answer	exerciseAnswer	Anzuzeigender Eintrag
setAnswerState	(id, isActive) => void	Meldet Auswahl/Abwahl an LinkingExercise
resetAnswer	boolean	Signal, die Auswahl zurückzusetzen
setInactive	boolean	Signal, das Element nach einem richtigen Paar zu deaktivieren

DragAndDropExercise

Zweck: Als Aufgabentyp "Drag-and-Drop" vorgesehen.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Hinweis: Die Datei ist aktuell leer und enthält weder Inhalt noch eine exportierte Komponente – dieser Aufgabentyp ist noch nicht umgesetzt.

ClickableImageExercise

Zweck: Als Aufgabentyp "anklickbares Bild" vorgesehen.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Hinweis: Die Datei ist aktuell leer und enthält weder Inhalt noch eine exportierte Komponente – dieser Aufgabentyp ist noch nicht umgesetzt.

8. Allgemeine UI-Bausteine

Diese Komponenten werden an mehreren Stellen der Anwendung wiederverwendet.

AlertBox

Zweck: Allgemeiner Bestätigungsdialog mit Titel, Nachricht und Ja-/Nein-Auswahl.

Prop	Typ	Bedeutung
titleText	string	Überschrift des Dialogs
messageText	string	Angezeigter Hinweistext
onClose	(result: boolean) => void	Callback mit der getroffenen Auswahl

PasswordAlertBox

Zweck: Dialog zur Eingabe eines neuen Passworts.

Prop	Typ	Bedeutung
onClose	(result: string) => void	Callback mit dem eingegebenen, bereinigten Passwort bzw. einem leeren String bei Abbruch

Verhalten

- Bereinigt die Eingabe über `sanitizeString`, bevor sie zurückgegeben wird.

UserIconAlertBox

Zweck: Dialog zur Auswahl eines neuen Profilbilds inklusive Hintergrundfarbe.

Prop	Typ	Bedeutung
userIcon	profileIconElement	Aktuell gewähltes Profilbild
onClose	(newId, newSrc, newColor) => void	Callback mit der neuen Auswahl

Verhalten

- Lädt die verfügbaren Icons über `getIconsData()` und zeigt sie zur Auswahl an.

ErrorPage

Zweck: Inhalt der 404-Fehlerseite mit Hinweistext.

Diese Komponente hat keine Props; sie bezieht ihre Daten direkt aus lib/ (z. B. Session-Storage oder Mock-Daten).

Verhalten

- Navigiert bei Klick auf die Seite einen Schritt in der Browser-Historie zurück.

9. Hilfsmodule (lib/)

Diese Module enthalten keine Benutzeroberfläche, sondern Typdefinitionen und Funktionen, die von mehreren Komponenten verwendet werden.

interfaceHandler.ts

Zweck: Definiert alle TypeScript-Schnittstellen (Typen) für Daten, die zwischen Komponenten und der API ausgetauscht werden.

Typ	Verwendet für
roomList / roomListItem	Liste aller Lernräume (Name, Vorschaubild, Beschreibung)
roomContent / roomTopic / roomTopicExercise / exerciseQuestion / exerciseAnswer	Vollständiger Inhalt eines Lernraums mit Themen, Aufgaben und Antworten
profileContent / profileStudentsInfo / profileRoomsProgress	Profildaten inklusive Schüler- und Raum-Fortschritt
profileIconContent / profileIconElement	Verfügbare bzw. gewählte Profilbilder
requestStudentManagement / requestAccountManagement	Anfragen zur Schüler- bzw. Konto-Verwaltung an die API
createdStudentsInfo / newStudentInfo	Zugangsdaten neu angelegter Schüler
feedbackContent	Feedback-Text für die Über-uns-Seite
webToken / loginSignalContent	Anmeldetoken bzw. Anmeldedaten
roomTopicStatus	Meldung, dass ein Thema abgeschlossen wurde

globalVars.ts

Zweck: Zentrale Konstanten (Routen-Pfade, API-Adresse) sowie Verwaltung des Anmeldestatus.

Methode	Beschreibung
isLoggedIn()	Prüft, ob ein gültiges, nicht abgelaufenes Anmeldetoken im Session-Storage vorliegt (Gültigkeit: 24 Stunden).
getLoginToken()	Liest das gespeicherte Anmeldetoken aus dem Session-Storage.
setLoginState(token)	Speichert ein neues Anmeldetoken bzw. löscht es und sendet ein Abmelde-Signal, wenn null übergeben wird.

navigationHandler.ts

Zweck: Stellt eine einheitliche Navigationsfunktion für alle Komponenten bereit.

Methode	Beschreibung
navigateToPage()	Liefert eine Funktion, die zu einer übergebenen Seiten-URL navigiert und den Bildlauf zurücksetzt.

sessionStorageHandler.ts

Zweck: Kapselt den Zugriff auf den Session-Storage des Browsers.

Methode	Beschreibung
saveToSessionStorage(key, data)	Speichert einen Wert unter einem Schlüssel.
getFromSessionStorage(key)	Liest einen Wert; liefert einen leeren String, wenn nichts gespeichert ist.

stringHandler.ts

Zweck: Bereinigt und kodiert Texteingaben, bevor sie weiterverarbeitet oder an die API gesendet werden.

Methode	Beschreibung
sanitizeString(input)	Entfernt überflüssige Leerzeichen und unsichtbare Zeichen und escaped HTML-relevante Sonderzeichen.
base64Encode(input) / base64Decode(input)	Kodieren bzw. Dekodieren eines Texts in Base64.

Methode	Beschreibung
processForAPISend(input) / processForAPIRecieve(input)	Kombinieren Bereinigung und Base64-Kodierung für den Versand an bzw. Empfang von der API.

9.1 API-Anbindung

apiHandler.ts

Zweck: Stellt die Funktionen bereit, über die Daten von der API abgerufen bzw. an die API gesendet werden.

Methode	Beschreibung
recieveRoomsList()	Liefert die Liste aller Lernräume.
recieveRoomContent(roomName)	Liefert den vollständigen Inhalt eines Lernraums.
recieveProfileData()	Liefert die Profildaten des angemeldeten Benutzers.
sendLoginData(username, password)	Sendet Anmeldedaten und liefert ein Anmeldetoken.
sendSignupData(username, password)	Sendet Registrierungsdaten.
sendLogoutSignal()	Meldet das Abmelden an die API.
addNewStudents(count)	Legt eine bestimmte Anzahl neuer Schüleraccounts an.
manageUserAcc(managementType, userName)	Löscht einen Schüleraccount oder setzt dessen Passwort zurück.
manageOwnAcc(managementType, option)	Ändert das eigene Profilbild oder Passwort bzw. löscht den eigenen Account.
sendFeedback(feedbackText)	Sendet einen Feedback-Text.
sendTopicStatus(roomID, topicID, complete)	Meldet den Abschluss eines Themas.

Hinweis: Alle genannten Funktionen liefern aktuell fest hinterlegte Testdaten statt echte Serverantworten. Der auskommentierte Code für den jeweiligen echten API-Aufruf ist bereits vorhanden, aber noch nicht aktiv.

dataHandler.ts

Zweck: Liegt zwischen den Komponenten und apiHandler.ts: hält Daten im Session-Storage vor und wertet sie für die Komponenten auf.

Methode	Beschreibung
getRoomListItems() / getRoomNames() / getRoomThumbnail(index)	Liefern die Raumliste bzw. einzelne Angaben daraus, zwischengespeichert im Session-Storage.
getRoomContent(roomName)	Liefert den Rauminhalt oder einen Platzhalter-Inhalt, falls der Raum nicht gefunden wird.
getProfileData()	Liefert die Profildaten des Benutzers.
getIconsData() / getIconSrc(id)	Liefern die Liste verfügbarer Profilbilder bzw. den Pfad zu einem einzelnen Bild.

9.2 Konstanten

AnswerState.ts

Zweck: Definiert die möglichen Zustände einer Antwort in Aufgaben.

Methode	Beschreibung
UNANSWERED / CORRECT / WRONG	Numerische Konstanten (0, 1, 2) für den Status einer Antwort.

ManagementType.ts

Zweck: Definiert die möglichen Verwaltungsaktionen für Schüler- bzw. eigene Accounts.

Methode	Beschreibung
UserManagementType	ADD, DELETE, PASSWORD_RESET – Aktionen für Schüleraccounts.
AccountManagementType	CHANGE_PROFILE_ICON, CHANGE_PASSWORD, DELETE_ACCOUNT – Aktionen für den eigenen Account.

10. Projekt-Setup und Einstiegspunkt

`main.tsx`

Zweck: Einstiegspunkt der Anwendung. Bindet die Komponente App in das HTML-Element mit der Id root ein.

`App.tsx`

Zweck: Definiert das Routing der gesamten Anwendung (siehe Tabelle in Kapitel 2) über React Router.

`vite.config.ts`

Zweck: Grundkonfiguration von Vite mit dem React-Plugin für die Entwicklung und den Produktions-Build.